



Tecnicatura Superior en Desarrollo de Software

Anexo 05g — `OperationResult<T>`

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat |
Ciclo Lectivo 2026

Documento reemplazado:

Este anexo quedó superado por **Anexo 07a — `OperationResult<T>`**, que es el que está asociado al Módulo 07 en la bitácora del curso y el que se mantiene actualizado. Se conserva este archivo por su valor histórico (tiene PDF ya entregado en un ciclo anterior), pero no debe usarse como referencia vigente — puede citar nombres de métodos u organización de archivos que ya cambiaron.

¿Qué es `OperationResult<T>`?

`OperationResult<T>`

Objeto de resultado
estandarizado

Capa: Application

`OperationResult<T>` es un objeto que encapsula el resultado de una operación. En lugar de lanzar una excepción cuando algo falla — o devolver `null` cuando no hay resultado — la operación siempre devuelve un `OperationResult` que puede representar dos estados posibles: **éxito** con un valor de tipo `T`, o **fallo** con una lista de errores descriptivos.

Este patrón — también conocido como *Result Type* o *Discriminated Union* en otros lenguajes — hace que el flujo de la aplicación sea explícito: el llamador siempre sabe que debe verificar si la operación fue exitosa antes de usar el valor devuelto.

Estados posibles:

- **Ok(value)** — éxito con dato
- **Fail(errors)** — fallo con errores
- `IsSuccess` — indica el estado
- `Value` — dato si exitoso
- `Errors` — lista si fallido

El problema que resuelve:

Cuando un UseCase termina — ya sea con éxito o con un error esperado — necesita comunicar ese resultado al OutputPort. Sin `OperationResult`, esa comunicación se hace con múltiples métodos separados, variables auxiliares o excepciones usadas como control de flujo.

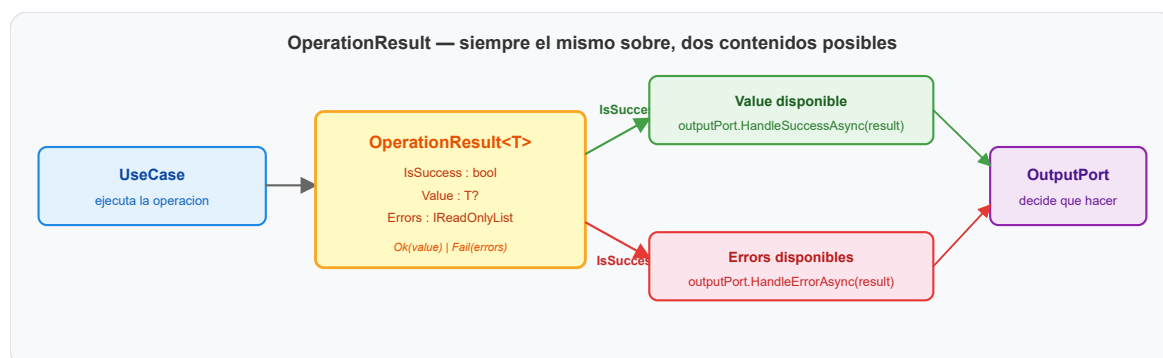
`OperationResult<T>` unifica esa comunicación en un único objeto: el UseCase construye el resultado y lo pasa al OutputPort, que decide qué hacer según el estado.

Analogía del mundo real

Imaginá que pedís un análisis de laboratorio. Unos días después recibís un sobre. Dentro del sobre puede haber dos cosas: el resultado con los valores del análisis, o una nota que dice "muestra insuficiente — repetir extracción".

En ambos casos recibís **el mismo sobre** — la misma estructura. Antes de leer el resultado, mirás si hay una nota de error. Si no la hay, leés los valores. Si la hay, leés el motivo y tomás acción.

`OperationResult<T>` es ese sobre: siempre llega, siempre tiene la misma estructura, y el receptor sabe exactamente cómo manejarlo — sin importar si trajo buenas o malas noticias.



Características esenciales

Estructura de la clase

`OperationResult<T>` tiene constructor privado y solo se puede crear mediante los métodos estáticos `Ok()` y `Fail()`. Esto garantiza que nunca exista un resultado en estado indeterminado — siempre es éxito con valor o fallo con errores.

```
1 // clase sellada – no se puede heredar
2 public sealed class OperationResult<T>
3 {
4     // constructor privado – solo Ok() y Fail() pueden crear instancias
5     private OperationResult(bool isSuccess, T? value, IReadOnlyList<string> errors)
6 }
```

```
6      {
7          IsSuccess = isSuccess;
8          Value      = value;
9          Errors     = errors;
10     }
11
12     // true si la operación terminó con éxito
13     public bool IsSuccess { get; }
14
15     // valor producido – solo tiene sentido si IsSuccess es true
16     public T? Value { get; }
17
18     // errores – solo tienen sentido si IsSuccess es false
19     public IReadOnlyList<string> Errors { get; }
20
21     // crea un resultado exitoso con el valor producido
22     public static OperationResult<T> Ok(T value)
23         => new(true, value, Array.Empty<string>());
24
25     // crea un resultado fallido con la lista de mensajes de error
26     public static OperationResult<T> Fail(IReadOnlyList<string> errors)
27         => new(false, default, errors);
28
29     // sobrecarga conveniente para un solo mensaje de error
30     public static OperationResult<T> Fail(string error)
31         => new(false, default, new[] { error });
32 }
```

Uso en un UseCase

El UseCase produce un `OperationResult` al final de su ejecución y lo pasa al `OutputPort`. El `OutputPort` inspecciona `IsSuccess` y decide cómo notificar al llamador.

```
1  public async Task ExecuteAsync(RegisterBookRequest request)
2  {
3      // paso 1 – validación de precondiciones
4      var errors = _validator.Validate(request).ToList();
5      if (errors.Count > 0)
6      {
7          // construye resultado fallido y notifica
8          var failed = OperationResult<RegisterBookResponse>.Fail(errors.Select(e => e.ErrorMessage).ToList());
9          await _outputPort.HandleAsync(failed);
10         return;
11     }
```

```
12
13     // paso 2 – dominio y persistencia
14     Book book = Book.Create(request.Title, request.Author);
15     await _repository.AddAsync(book);
16
17     // paso 3 – construye resultado exitoso y notifica
18     var response = new RegisterBookResponse { BookId = book.Id };
19     var success = OperationResult<RegisterBookResponse>.Ok(response);
20     await _outputPort.HandleAsync(success);
21 }
```

OperationResult sin valor — operaciones void

Algunas operaciones no producen un valor de retorno — solo indican si tuvieron éxito o fallaron. Para estos casos existe la versión no genérica `OperationResult` con un tipo marcador. La solución más simple es usar `OperationResult<bool>` donde el valor de éxito es simplemente `true`.

```
1 // operación que no produce un valor específico
2 // se usa bool como marcador – true indica éxito
3 var result = OperationResult<bool>.Ok(true);
4
5 // o con un tipo Unit explícito si el proyecto lo define
6 public readonly struct Unit { } // tipo vacío – sin datos
7 var result = OperationResult<Unit>.Ok(new Unit());
```

¿Por qué se usa?

Cuando una operación puede fallar de forma esperada — validación, unicidad, regla de negocio — hay tres alternativas sin `OperationResult`:

Alternativas sin `OperationResult` — y sus problemas:

Excepción: correcto para errores inesperados, pero incorrecto para flujos esperados. Usar excepciones como control de flujo es costoso en rendimiento y confunde al llamador.

Retornar null: obliga al llamador a adivinar si null significa "no encontrado", "error" o "aún no disponible". No expresa la causa del fallo.

Variables auxiliares: el método modifica variables externas o recibe parámetros `out` — dificulta la lectura y el testeado del código.

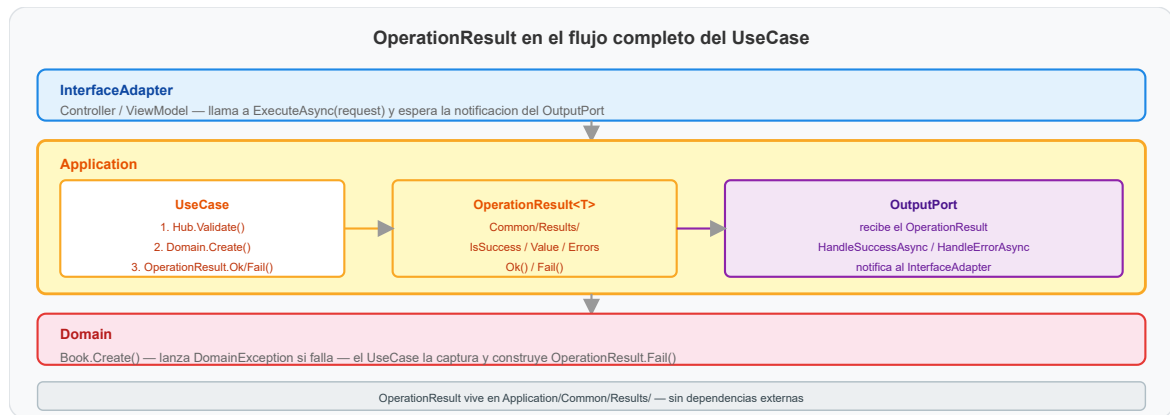
`OperationResult<T>` resuelve los tres problemas: el fallo es explícito, la causa está incluida en el objeto y el llamador no puede ignorar el estado sin leer `IsSuccess`.

Beneficios concretos:

1. **Flujo explícito:** el llamador siempre verifica `IsSuccess` — no puede olvidarse de manejar el error.
2. **Causa incluida:** `Errors` describe qué falló — sin necesidad de capturar excepciones ni interpretar códigos.
3. **Testeable:** el test verifica el estado del resultado directamente — sin capturar excepciones ni mockear efectos secundarios.

¿Cómo encaja en la arquitectura?

`OperationResult<T>` vive en la capa **Application**, en la carpeta `Common/`. Es compartido por todos los UseCases — no tiene dependencias externas, solo tipos del propio lenguaje.



Ubicación en el proyecto:

CommerX.Application/Common/Results/OperationResult.cs

Es el único archivo — sin interfaces ni implementaciones adicionales. Todos los UseCases lo usan directamente.

Marco teórico

Ejemplo completo ejecutable en Main()

El siguiente ejemplo implementa `OperationResult<T>` completo y lo usa en un UseCase simulado. No requiere dependencias externas — todo funciona en un programa de consola.

```

1  // implementación completa de OperationResult
2  public sealed class OperationResult<T>
3  {
4      private OperationResult(bool isSuccess, T? value, IReadOnlyList<string> errors)
5      {
6          IsSuccess = isSuccess;
7          Value     = value;
8          Errors    = errors;
9      }
10     public bool      IsSuccess { get; }
11     public T?        Value     { get; }
12     public IReadOnlyList<string> Errors { get; }

```

```
13     public static OperationResult<T> Ok(T v) => new(true, v, Array.Empty<string>());
14     public static OperationResult<T> Fail(string e) => new(false, default, new[]
15     { e });
16 }
17 // simulacion de un servicio que puede fallar
18 static OperationResult<string> BuscarLibro(string isbn)
19 {
20     // simula que el ISBN "978-0" no existe
21     if (isbn == "978-0")
22         return OperationResult<string>.Fail("Libro no encontrado.");
23
24     return OperationResult<string>.Ok($"Titulo para ISBN {isbn}");
25 }
26
27 // Main() - el llamador siempre verifica IsSuccess
28 var r1 = BuscarLibro("978-0");
29 if (!r1.IsSuccess)
30     Console.WriteLine($"Error: {r1.Errors[0]}"); // Error: Libro no encontrado.
31
32 var r2 = BuscarLibro("978-3-16-148410-0");
33 if (r2.IsSuccess)
34     Console.WriteLine($"Encontrado: {r2.Value}"); // Encontrado: Titulo para ISBN
35     978-3-...
```

Errores comunes

Error 1 — acceder a Value sin verificar IsSuccess

Si la operación falló, `Value` es `null` o `default`. Acceder a sus miembros sin verificar `IsSuccess` primero produce una `NullReferenceException`.

ERROR COMUN

Value usado directamente sin verificar IsSuccess.

```
1 var result = BuscarLibro("isbn-invalido");
2 // MAL: si IsSuccess es false, Value es null
3 Console.WriteLine(result.Value.Length); // NullReferenceException
```

BUENA PRACTICA

Siempre verificar `IsSuccess` antes de usar `Value`.

```
1 var result = BuscarLibro("isbn-invalido");
2 if (!result.IsSuccess) return;
3 // a partir de aquí, Value está garantizado
4 Console.WriteLine(result.Value!.Length);
```

Error 2 — usar `OperationResult` para errores inesperados

`OperationResult` es para flujos esperados — fallos que el sistema anticipa y comunica al usuario. Los errores inesperados — base de datos caída, timeout de red, bug de programación — deben lanzar excepciones, no encapsularse en un `OperationResult`.

ERROR COMUN

Excepción de infraestructura capturada y convertida en `Fail()`.

```
1 try
2 {
3     await _repo.AddAsync(book);
4     return OperationResult<bool>.Ok(true);
5 }
6 catch (SqlException ex)
7 {
8     // MAL: error de infraestructura no es un flujo esperado
9     return OperationResult<bool>.Fail(ex.Message);
10 }
```

BUENA PRACTICA

Errores inesperados propagan la excepción — el middleware global los maneja.

```
1 // sin try/catch — la excepción sube al middleware
2 await _repo.AddAsync(book);
3 return OperationResult<bool>.Ok(true);
4 // SqlException no es un flujo esperado — el middleware la captura
```

Qué NO hacer**No usar OperationResult como contenedor de excepciones de dominio**

Las `DomainException` se lanzan desde los Value Objects cuando una invariante es violada. El UseCase las captura y las convierte en `OperationResult<T>.Fail()` si corresponde comunicarlas al usuario. Pero el VO nunca debe retornar un `OperationResult` — eso rompería su patrón Fail Fast.

QUE NO HACER

Value Object retorna OperationResult — rompe Fail Fast y mezcla capas.

```
1 // MAL: VO de dominio retorna OperationResult de Application
2 public static OperationResult<BookTitle> Create(string value)
3 {
4     if (string.IsNullOrEmpty(value))
5         return OperationResult<BookTitle>.Fail("Titulo invalido.");
6     return OperationResult<BookTitle>.Ok(new BookTitle(value));
7 }
```

BUENA PRACTICA

VO lanza DomainException. El UseCase la captura y construye el Fail().

```
1 // VO - Fail Fast con DomainException
2 public static BookTitle Create(string value)
3 {
4     if (string.IsNullOrEmpty(value))
5         throw new InvalidBookTitleException(value);
6     return new BookTitle(value);
7 }
8 // UseCase captura y convierte si corresponde comunicarlo
```

Resumen — reglas de OperationResult:

- Solo para flujos esperados — no para errores de infraestructura.
- Siempre verificar `IsSuccess` antes de acceder a `Value`.
- Los Value Objects usan DomainException — nunca retornan OperationResult.
- El UseCase construye el resultado — el OutputPort decide cómo comunicarlo.